VSEPR-SIM Theoretical Division

TOP SECRET — Internal Development Document

Kernel V4

Bilinear Projection Form for Radial Density

Corrected separability analysis | Gradient/numerical caching form | Matrix presolve form | Generalised multi-channel projection engine

VSEPR-SIM Development Day 64

Contents

§1 Correction: Separability in the Gradient Form

1.1 The claim that needed fixing

An earlier version of the derivation stated that the gradient form *destroys* separability of the density into spatial and temporal parts, and therefore cannot cache spatial structure. This is too strong.

The gradient form hides the clean matrix structure, but separability is preserved. Define the **base density** and **difference density**:

$$B(\mathbf{r}) = \tfrac{1}{2}\left[R_1^2(\mathbf{r}) + R_2^2(\mathbf{r})\right] \tag{1}$$

$$D(\mathbf{r}) = \tfrac{1}{2}\left[R_1^2(\mathbf{r}) - R_2^2(\mathbf{r})\right] \tag{2}$$

Then the full density is:

$$\boxed{\rho(\mathbf{r}, t) = B(\mathbf{r}) + e_z(t)\, D(\mathbf{r})} \tag{3}$$

This is already cacheable. Spatial derivatives separate cleanly:

$$\nabla\rho(\mathbf{r}, t) = \nabla B(\mathbf{r}) + e_z(t)\, \nabla D(\mathbf{r}) \tag{4}$$

since $e_z(t)$ carries no spatial dependence.

Corrected claim:

> *The gradient form does not destroy separability; it merely hides the clean matrix structure. The matrix form makes the separability* explicit *and* generalisable*. The two claims are complementary, not equivalent.*

§2 Three Equivalent Forms

2.1 Form 1 — Decomposed projection form (readable)

$$\rho(\mathbf{r}, t) = \tfrac{1}{2}\left[(1 + e_z(t))\, R_1^2(\mathbf{r}) + (1 - e_z(t))\, R_2^2(\mathbf{r})\right] \tag{5}$$

This is the most readable form. It shows directly that the two radial projection channels R_1^2 and R_2^2 are weighted by $(1 \pm e_z(t))/2$, which sum to 1 when $e_z(t) = 0$ (equal weight) and collapse to a single channel when $e_z(t) = \pm 1$.

2.2 Form 2 — Expanded modulation form (numerical / caching)

Expand (**??**):

$$\rho(\mathbf{r}, t) = \underbrace{\tfrac{1}{2}R_1^2(\mathbf{r}) + \tfrac{1}{2}R_2^2(\mathbf{r})}_{B(\mathbf{r})} + e_z(t)\, \underbrace{\tfrac{1}{2}\left[R_1^2(\mathbf{r}) - R_2^2(\mathbf{r})\right]}_{D(\mathbf{r})} \tag{6}$$

giving (**??**).

Precomputed cache per grid point $\mathbf{r}_i$:

$$\{B_i,\ D_i,\ \nabla B_i,\ \nabla D_i\} \tag{7}$$

Runtime evaluation at time t:

$$\rho_i(t) = B_i + e_z(t)\, D_i \tag{8}$$
$$\nabla\rho_i(t) = \nabla B_i + e_z(t)\, \nabla D_i \tag{9}$$

This is the computationally useful statement: spatial precomputation is done once; each timestep costs two flops per grid point (one multiply, one add) for ρ, and $2d$ flops for $\nabla\rho$ in d spatial dimensions.

2.3 Form 3 — Matrix presolve form (dot-product runtime)

Write the radial channel vector and time-weight vector:

$$\mathbf{R}(\mathbf{r}) = \begin{bmatrix} R_1^2(\mathbf{r}) \\ R_2^2(\mathbf{r}) \end{bmatrix}, \qquad \mathbf{w}(t) = \tfrac{1}{2}\begin{bmatrix} 1 + e_z(t) \\ 1 - e_z(t) \end{bmatrix} \tag{10}$$

Then:

$$\boxed{\rho(\mathbf{r}, t) = \mathbf{R}(\mathbf{r})^\top \mathbf{w}(t)} \tag{11}$$

Runtime cost for a single preloaded grid point: $O(1)$.

§3 Generalised Bilinear Projection Form

For n radial channels and $m + 1$ temporal state channels, define:

$$\mathbf{R}(\mathbf{r}) = \begin{bmatrix} R_1^2(\mathbf{r}) \\ R_2^2(\mathbf{r}) \\ \vdots \\ R_n^2(\mathbf{r}) \end{bmatrix} \in \mathbb{R}^n, \qquad \mathbf{e}(t) = \begin{bmatrix} 1 \\ e_1(t) \\ e_2(t) \\ \vdots \\ e_m(t) \end{bmatrix} \in \mathbb{R}^{m+1} \tag{12}$$

Let $\mathbf{M} \in \mathbb{R}^{n\times(m+1)}$ be the **channel mixing matrix** that maps temporal state channels into radial-channel weights. Then:

$$\boxed{\rho(\mathbf{r}, t) = \mathbf{R}(\mathbf{r})^\top \mathbf{M}\, \mathbf{e}(t)(t)} \tag{13}$$

The two-channel case (**??**) is recovered with:

$$\mathbf{M} = \tfrac{1}{2}\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \qquad \mathbf{e}(t)(t) = \begin{bmatrix} 1 \\ e_z(t) \end{bmatrix} \tag{14}$$

Why this form matters for code architecture:

- $\mathbf{R}(\mathbf{r})$ and $\mathbf{M}$ are fully spatial / static — precomputed once before the time loop.
- $\mathbf{e}(t)(t)$ is a small vector updated each step from the temporal state kernel.
- The inner product $\mathbf{R}(\mathbf{r})^\top \mathbf{M}\, \mathbf{e}(t)$ at each grid point is a dense matrix-vector product on a small system ($n \times (m + 1)$ is typically 2–8 × 2–8).
- This scales cleanly from a 2-channel toy case to a multi-channel projection engine without restructuring.

§4 Presolve Pipeline

4.1 Offline presolve (done once)

For each spatial grid point $\mathbf{r}_i$, $i = 1, \ldots, N$:

$$\mathbf{R}_i = \begin{bmatrix} R_1^2(\mathbf{r}_i) \\ R_2^2(\mathbf{r}_i) \end{bmatrix} \tag{15}$$

Assemble the grid matrix:

$$R_{\text{grid}} \in \mathbb{R}^{N \times n}, \quad [R_{\text{grid}}]_{i,k} = R_k^2(\mathbf{r}_i) \tag{16}$$

Compute and cache $R_{\text{grid}}\mathbf{M}$ (static, shape $N \times (m+1)$).

4.2 Runtime (each timestep)

Update $\mathbf{e}(t)(t)$ from the temporal state kernel, then:

$$\boldsymbol{\rho}(t) = (R_{\text{grid}}\mathbf{M})\, \mathbf{e}(t)(t) \tag{17}$$

Cost analysis:

Operation	Cost	Notes
Precompute $R_{\text{grid}}\mathbf{M}$	$O(Nn(m+1))$	Once, offline
Update $\mathbf{e}(t)(t)$	$O(m)$	Temporal kernel
Evaluate $\boldsymbol{\rho}(t)$	$O(N(m+1))$	Matrix-vector product
Single-point lookup	$O(m+1) \approx O(1)$	After presolve

For the two-channel case $n = 2$, $m = 1$: runtime evaluation is $O(2N)$, or $O(1)$ per preloaded point.

§5 Reference Implementation

5.1 Python (prototype)

```python
import numpy as np

class KernelV4:
  """
  Bilinear projection engine for radial density.
  Two-channel case: rho(r,t) = R(r)^T w(t)
  """

  def __init__(self, R1_grid: np.ndarray, R2_grid: np.ndarray):
    """
    Parameters
    ----------
    R1_grid : (N,) array of R1^2 values on the spatial grid
    R2_grid : (N,) array of R2^2 values on the spatial grid
    """
    # Spatial cache  [N x 2]
    self.R_grid = np.stack([R1_grid, R2_grid], axis=1)

    # Channel mixing matrix M  [2 x 2]
```

```python
    self.M = 0.5 * np.array([[1.0,  1.0],
                             [1.0, -1.0]])

    # Presolve:  R_grid @ M  is [N x 2], computed once
    self._RM = self.R_grid @ self.M

  def weight_vector(self, ez: float) -> np.ndarray:
    """Temporal weight vector e(t) = [1, ez]."""
    return np.array([1.0, ez])

  def density(self, ez: float) -> np.ndarray:
    """Runtime evaluation: rho(t) = (R_grid M) e(t).  Cost: O(2N)."""
    return self._RM @ self.weight_vector(ez)

  def density_single(self, i: int, ez: float) -> float:
    """Single-point lookup after presolve.  Cost: O(1)."""
    return float(self._RM[i] @ self.weight_vector(ez))


# --- Example usage ------------------------------------------------------
if __name__ == "__main__":
  N = 1000
  r = np.linspace(0.0, 5.0, N)

  # Toy radial functions: hydrogen-like s-orbitals
  R1 = np.exp(-r)              # R1^2 ~ exp(-2r), but use R1 for clarity
  R2 = r * np.exp(-r / 2.0)

  kernel = KernelV4(R1**2, R2**2)

  # Time evolution: ez sweeps from -1 to 1
  for ez in np.linspace(-1.0, 1.0, 5):
    rho = kernel.density(ez)
    print(f"ez={ez:+.2f}  rho[0]={rho[0]:.6f}  rho[-1]={rho[-1]:.6f}")
```

Listing 1: Kernel V4 two-channel presolve (Python)

5.2 C++ kernel hook (production sketch)

```cpp
#pragma once
#include <span>
#include <vector>
#include <array>
#include <cstddef>

namespace ikk {

/// Two-channel bilinear projection kernel.
/// rho(r,t) = R_grid * M * e(t)
/// Presolve: cache R_grid * M offline.  Runtime: O(2N) matvec.
class KernelV4 {
public:
  /// Presolve constructor.
```

```cpp
  /// r1sq, r2sq: R1^2 and R2^2 evaluated on the spatial grid, length N.
  KernelV4(std::span<const double> r1sq,
           std::span<const double> r2sq)
  {
    const std::size_t N = r1sq.size();
    rm_.resize(N);                    // rm_[i] = {B_i, D_i}

    for (std::size_t i = 0; i < N; ++i) {
      rm_[i][0] = 0.5 * (r1sq[i] + r2sq[i]);    // B_i
      rm_[i][1] = 0.5 * (r1sq[i] - r2sq[i]);    // D_i
    }
  }

  /// Full grid evaluation.  rho_out must have length N.
  /// Cost: O(2N)
  void density(double ez, std::span<double> rho_out) const noexcept {
    for (std::size_t i = 0; i < rm_.size(); ++i)
      rho_out[i] = rm_[i][0] + ez * rm_[i][1];
  }

  /// Single-point lookup after presolve.  Cost: O(1).
  [[nodiscard]]
  double density_at(std::size_t i, double ez) const noexcept {
    return rm_[i][0] + ez * rm_[i][1];
  }

private:
  /// Cached (R_grid * M): rm_[i] = {B_i, D_i}
  std::vector<std::array<double, 2>> rm_;
};

} // namespace ikk
```

Listing 2: Kernel V4 presolve hook (C++23)

§6 Summary

Form	Equation	Best for	Cache
Decomposed projection	(??)	Reading, intuition	No
Expanded modulation	(??)	Gradient / finite-diff	$B_i, D_i, \nabla B_i, \nabla D_i$
Matrix presolve	(??)	Dot-product runtime	$\mathbf{R}_i^\top$
Generalised bilinear	(??)	Multi-channel engine	$R_{\text{grid}}\mathbf{M}$

All four forms are algebraically equivalent for the two-channel case. The generalised bilinear form (??) is the target for production code because it is:

- **Statically typed by shape**: $R_{\text{grid}} \in \mathbb{R}^{N\times n}$, $\mathbf{M} \in \mathbb{R}^{n\times(m+1)}$, $\mathbf{e}(t) \in \mathbb{R}^{m+1}$.
- **BLAS-friendly**: the presolve is a single DGEMM; the runtime step is a DGEMV.
- **Channel-count agnostic**: adding a new radial or temporal channel is a shape change in R_{grid} or $\mathbf{M}$, not a restructuring of the kernel.

Kernel V4 — VSEPR-SIM Theoretical Division. Humanity survives another algebraic rearrangement.